

Cross-Site Request Forgery (CSRF) Vulnerabilities

CSRF tricks an authenticated user's browser into sending unintended requests to a vulnerable site. The browser automatically includes cookies (session tokens), so the target site thinks the request is legitimate.

Unlike XSS (which injects code), CSRF uses legitimate requests but forges the user's intent. It targets state-changing actions (POST, sometimes GET).

History & Evolution

- **Past:** Widespread in early banking and forum apps. Famous cases: 2008 Netflix password change via email link. Developers used predictable cookies without extra checks.
- **Present:** Declining but persistent. Recent 2025–2026 examples:
 - CVE-2025-67626 (WordPress WP SEO Search plugin CSRF).
 - Cisco Unified Communications Manager CSRF (Sep 2025).
 - Yokogawa FAST/TOOLS ICS vulnerability (CVE-2025-66595, early 2026).
 - Multiple WordPress plugin issues (e.g., Image Optimizer, SurveyJS). Still appears in CMS plugins and industrial systems.
- **Future Trends:** Browser defaults (SameSite=Lax cookies since ~2020) block most cross-site POSTs. CSRF will fade further but linger in APIs, SPAs needing custom headers, or misconfigured legacy apps.

How CSRF Works (Step-by-Step)

1. User logs into vulnerable.site (receives session cookie).
2. User visits evil.com (or clicks malicious link/email).
3. evil.com hosts forged request (hidden form or img tag).
4. Browser sends request to vulnerable.site with user's cookies.
5. Server processes as legitimate → action executes.

Differences from Similar Vulnerabilities

Vulnerability	Requires Auth Session	Executes Code?	User Interaction	Primary Defense	Example Impact
CSRF	Yes	No (forged req)	Yes (visit/click)	Anti-CSRF tokens, SameSite	Unauthorized money transfer
XSS	No	Yes (JS)	Varies	Output encoding, CSP	Session steal, keylogging
Clickjacking	Yes	No	Yes (click)	X-Frame-Options, CSP frame-ancestors	Trick into clicking hidden button
Open Redirect	No	No	Yes	Whitelist redirects	Phishing aid
SSRF	No (server-side)	No	No	Input validation	Internal network access

CSRF uniquely exploits browser cookie auto-send behavior.

Impacts & Real-World Effects

- Unauthorized transactions (money transfer).
- Account changes (email/password reset).
- Data modification/deletion.
- Privilege escalation in admin panels.
- In ICS (like Yokogawa 2026 case): Potential operational disruption.

Current Tools:

- **Manual:** Burp Suite (built-in CSRF PoC generator), OWASP ZAP.
- **Automated:** Nuclei templates, Burp Scanner.
- **Testing:** Browser dev tools to remove SameSite and test legacy behavior.

Examples:

1. **PortSwigger Web Academy** – Top-tier CSRF labs (basic → advanced token bypass → SameSite understanding).
2. **DVWA** – CSRF module (Low: no protection → High: token required).
3. Generate PoC in Burp and test on legal labs.

Example from DVWA low-level lab:

The screenshot displays the DVWA web application interface in a browser window. The address bar shows `localhost/dvwa/vulnerabilities/csrf/`. The browser's tab bar includes several open tabs, with the active one being "Unsupervised Learn...". The DVWA logo is visible in the top right corner of the application header.

On the left side, there is a vertical navigation menu with various security modules. The "CSRF" module is currently selected and highlighted in green. Other modules listed include Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, Open HTTP Redirect, Cryptography, API, DVWA Security, PHP Info, and About.

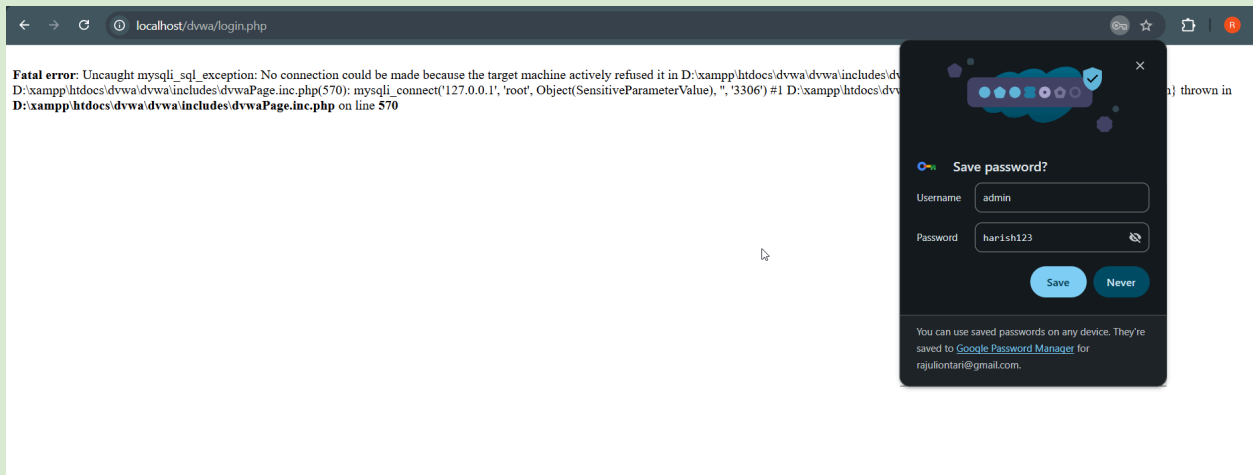
The main content area is titled "Vulnerability: Cross Site Request Forgery (CSRF)". It features a form for changing the admin password. The form includes a "Test Credentials" button, input fields for "New password:" (containing "harish123") and "Confirm new password:" (containing "harish123"), and a "Change" button.

Below the form, a note states: "Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected."

Under the heading "Announcements:", there is a list of browser versions: Chromium, Edge, and Firefox.

A paragraph explains an alternative attack method: "As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start."

The section "More Information" provides three links: <https://owasp.org/www-community/attacks/csrf>, <https://www.cgisecurity.com/csrf-faq.html>, and https://en.wikipedia.org/wiki/Cross-site_request_forgery.



Here, I used the beefree tool to generate a fake app of a website so that I could send an email to anyone and make them click the link, which added malicious content. So that we can gather their credentials. CSRF here performs if the user already signed in to this webapp.

